Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»

Кафедра СМ-6 «Ракетные и импульсные системы»

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине
«Внутренняя баллистика ствольных систем»

«Markdown, LaTeX и воспроизводимый отчёт по прямому баллистическому расчёту»

Студент: Барышев Савва
Группа: СМ6-72
Преподаватель: Товарнов М.С.

Москва
2026

ВВЕДЕНИЕ

Цель работы — реализовать воспроизводимый прямой расчёт внутренней баллистики для варианта № 17, проверить вычисление критерия и всех ограничений, исследовать действие штрафной функции на допустимом и недопустимом решениях и подтвердить контролируемую обработку ошибки модели. Физические определения и параметры варианта приняты по учебно-методическому пособию [1], а расчёт выполняется квазиодномерной моделью `ozvb_lagrange` библиотеки `pyballistics` версии 1.2.1 [2]. Отчёт оформлен по сокращённому профилю ГОСТ 7.32-2017 [3].

Для проверки выбраны семь точек: основное допустимое и недопустимое решения, нерассчитываемая точка с неизвестной маркой пороха, отдельная диагностическая точка без достижения целевой скорости, близкая пара по разные стороны границы горячего дульного давления и точка с увеличенным нарушением этой границы. Такой набор позволяет раздельно проверить численный статус, отделение неполного целевого расчёта от полноценного решения, чистый критерий и штраф, а также подтвердить рост штрафа с величиной рассчитанного нарушения. Изменение скорости, среднебаллистического давления, полноты сгорания и штрафа представлено графически.

1 ИСХОДНЫЕ ДАННЫЕ И КРИТЕРИЙ

1.1 Параметры варианта

Рассматривается нарезное орудие калибра $d = 152{,}4$ мм с массой снаряда $q=43{,}56$ кг. Требуемая дульная скорость равна $v_{pm} = 700$ м/с. Для нарезного орудия площадь канала ствола определяется по формуле (1).

$$S = 0{,}26\pi d^2 = 0{,}018971128017 \text{ м}^2. \tag{1}$$

где S — площадь поперечного сечения канала ствола, м²; d — калибр орудия, м.

Основные постоянные расчёта приведены в таблице 1. Температуры $+20$ °C и $+50$ °C заданы как 293,15 К и 323,15 К соответственно.

Таблица 1 — Исходные данные варианта № 17

Величина	Обозначение	Значение
Калибр	d	0,1524 м
Масса снаряда	q	43,56 кг
Требуемая скорость	v_{pm}	700 м/с
Давление форсирования	p_0	30 МПа
Давление воспламенителя	$p_{ign,0}$	5 МПа
Коэффициент трения в нарезах	φ_1	1,02
Максимальное среднебаллистическое давление	p_m^{max}	350 МПа
Максимальная длина ведущей части	$l_{max} = 50d$	7,62 м
Максимальное относительное положение конца горения	$\eta_{e,max}$	0,6
Максимальное дульное давление при +50 °C	$p_{mz,+50}^{max}$	100 МПа

1.2 Критерий качества

Для варианта требуется минимизировать суммарный объём каморы и ведущей части канала ствола. Чистый критерий вычисляется по формуле (2) отдельно от штрафа.

$$W_{pm} = W_0 + Sl_m \longrightarrow \min. \tag{2}$$

где W_{pm} — объёмный критерий, м³; W_0 — начальный объём каморы, м³; l_m — длина ведущей части, необходимая для достижения 700 м/с, м.

Так как лабораторная работа не предусматривает оптимизацию, значение по формуле (2) используется только для сопоставления отдельных проверочных точек. Меньшее значение чистого критерия само по себе не означает допустимость решения.

2 МЕТОДИКА РАСЧЁТА И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

2.1 Квазиодномерная модель

Расчёт выполняется функцией `pyballistics.ozvb_lagrange`, которая решает прямую задачу в газодинамической постановке на лагранжевой сетке [2]. Используются 150 ячеек, число Куранта 0,9 и ограничение 100000 шагов. Для контроля аварийного роста локального давления расчёт останавливается при 700 МПа. Базовый расчёт при +20 °C содержит одновременно останов по $v_p=700$ м/с и защитный предел $x_p = 50d = 7{,}62$ м. Только штатный останов по v_p означает, что целевая длина l_m фактически найдена.

Если цель не достигнута до защитного предела, давления или лимита шагов, выполняется отдельное диагностическое интегрирование без останова по скорости до $2 \cdot 50d = 15{,}24$ м при тех же защитных ограничениях. Его конечные координата, скорость, причина останова и ряды сохраняются, но не объявляются l_m и не используются для W_{pm}, горячего расчёта, штрафа или классификации по ограничениям. После корректного определения l_m выполняется второй расчёт при +50 °C ровно до этой длины.

В каждом временном слое давление задано по ячейкам переменной длины. Среднебаллистическое давление вычисляется как конечно-объёмное пространственное среднее по формуле (3).

$$p_m(t) = \frac{\sum_{i=1}^{N} p_i(t)\,\Delta x_i(t)}{\sum_{i=1}^{N} \Delta x_i(t)}. \qquad (3)$$

где p_i — давление в i$-й ячейке, Па; Δx_i — длина ячейки, м; $N=150$ — число ячеек. Максимум ряда $p_m(t)$ до дульного среза образует p_m^{max}.

Ячейки лагранжевой сетки содержат одинаковые доли каждой навески. Поэтому интегральная доля сгоревшего заряда определяется по формуле (4).

$$\psi(t) = \frac{\sum_{j=1}^{M} \omega_j \overline{\psi_j}(t)}{\sum_{j=1}^{M} \omega_j}, \qquad \overline{\psi_j} = \frac{1}{N} \sum_{i=1}^{N} \psi_{ij}. \tag{4}$$

где M — число навесок; ω_j — масса \$j\$-й навески, кг; \$\psi_{ij}\$ — доля сгоревшего пороха этой навески в ячейке. Положение снаряда l_e при первом достижении $\psi = 1$ определяется линейной интерполяцией. Относительная координата конца горения равна \$\eta_e=l_e/l_m\$.

2.2 Ограничения и штрафная функция

Требуемая скорость 700 м/с задаёт положение дульного среза l_m только при фактическом останове базовой ветви по v_p. Условие достижения скорости не позднее $50d$ тем самым проверяется ограничением длины. Если базовый расчёт остановлен иначе, результат получает отдельный статус `incomplete`: его конечный путь — только диагностическая величина. Четыре ограничения варианта записаны формулой (5).

$$\begin{aligned} p_m^{max}(+20\,^\circ\mathrm{C}) &\le 350\ \text{МПа}, \\ l_m(+20\,^\circ\mathrm{C}) &\le 50d, \\ \eta_e(+20\,^\circ\mathrm{C}) &\le 0{,}6, \\ p_{mz}(+50\,^\circ\mathrm{C}) &\le 100\ \text{МПа}. \end{aligned} \tag{5}$$

Для совместного учёта величин разных размерностей вводится нормированное нарушение r_k. Для верхнего ограничения $g_k \le g_k^{max}$ и нижнего ограничения $g_k \ge g_k^{min}$ оно определяется по формуле (6).

$$r_k = \begin{cases} \max\left(0, \dfrac{g_k}{g_k^{max}} - 1\right), & g_k \le g_k^{max}, \\ \max\left(0, 1 - \dfrac{g_k}{g_k^{min}}\right), & g_k \ge g_k^{min}. \end{cases} \tag{6}$$

Штраф и штрафной критерий вычисляются по формулам (7) и (8).

$$P = C_P \sum_{k=1}^{4} r_k^2, \qquad C_P = 1\ \text{м}^3. \tag{7}$$

$$W_{pm}^{pen} = W_{pm} + P. \tag{8}$$

Квадратичная форма по формуле (7) имеет нулевое значение для допустимого решения и монотонно возрастает с увеличением любого нарушения. Коэффициент C_P возвращает штрафу размерность критерия. Для ошибки прямого расчёта чистый критерий не вычисляется, назначается отдельный штраф 10^6 м³ и статус `numerical_error`. Для статуса `incomplete` не назначаются ни критерий, ни штраф: это не численная ошибка и не полноценное недопустимое решение.

2.3 Обработка ошибок и воспроизводимость

Вызов `ozvb_lagrange` окружён `try-except`. Дополнительно обрабатывается возвращаемое библиотекой значение `stop_reason == "error"`. Успешные слои проверяются на наличие обязательных полей, непустых массивов, конечность чисел и корректность пространственной сетки. Поэтому нарушение ограничения получает статус `infeasible`, исключение, ошибка модели или некорректный результат — `numerical_error`, а отсутствие целевой длины или недостижение горячим расчётом общего среза — `incomplete`. После горячего расчёта проверяются не только отсутствие ошибки, но и `stop_reason == x_p` и совпадение последней координаты ряда с l_m. При раннем останове горячие ряды сохраняются как диагностика, но дульное давление, штраф и полная классификация не вычисляются.

Команда `uv run python main.py` выводит фактическую версию библиотеки, рассчитывает все сценарии и заново записывает пять рисунков в каталог `figures/`. Для ЧБ-печати кривые различаются одновременно типами линий и маркерами, а столбцы — штриховкой. Файлы сохраняются с разрешением 300 dpi и закрываются программно после записи.

3 ПРОВЕРОЧНЫЕ СЦЕНАРИИ И ЧИСЛЕННЫЕ РЕЗУЛЬТАТЫ

3.1 Выбор сценариев

Параметры семи проверочных точек приведены в таблице 2. Все рассчитываемые точки используют табличный порох 11/7. Основная допустимая точка имеет запас по всем условиям формулы (5), а основная недопустимая точка с повышенной плотностью заряжания нарушает два ограничения по давлению. Точка с массой 2,0 кг специально не достигает 700 м/с до $50d$ и проверяет разделение целевой и диагностической ветвей. В ошибочной точке намеренно задано отсутствующее в базе имя пороха, что проверяет контракт обработки ошибки, а не физическую допустимость.

Три дополнительные точки образуют доказательную серию при постоянном $W_0=0{,}035$ м³. Массы 11,495 и 11,500 кг дают близкую пару по разные стороны границы $p_{mz,+50} = 100$ МПа, а масса 11,600 кг увеличивает только это нарушение. Точки заданы явно; оптимизатор и автоматический поиск границы не применялись.

Таблица 2 — Параметры проверочных сценариев

Сценарий	Марка пороха	Масса заряда, кг	W_0, м³	Плотность заряжания, кг/м³	ω/q
Допустимый	11/7	10,0	0,035	285,714	0,229568
Цель не достигнута (диагностика)	11/7	2,0	0,035	57,143	0,045914
Граница снизу	11/7	11,495	0,035	328,429	0,263889
Граница сверху	11/7	11,500	0,035	328,571	0,264004
Увеличенное нарушение	11/7	11,600	0,035	331,429	0,266299
Недопустимый	11/7	11,5	0,025	460,000	0,264004
Ошибочный	`__unknown_powder__`	10,0	0,035	285,714	0,229568

Численный статус, причины остановов и конечные координаты приведены в таблице 3. У пяти полноценных физических точек базовая ветвь останавливается по `v_p`, а горячая — по общему срезу `x_p`. В диагностической точке цель не достигнута: защитный базовый останов и продолженный диагностический расчёт сохраняют разные конечные значения, а горячая ветвь не запускается. Неизвестная марка пороха приводит к контролируемому `stop_reason == "error"`.

Таблица 3 — Статусы и причины останова модели

Сценарий	Целевая ветвь +20 °C	Диагностика +20 °C	Горячая ветвь +50 °C	Итог
Допустимый	success, v_p, $x =$ 4,792 м	—	success, x_p, $x =$ 4,792 м	feasible
Цель не достигнута	success, x_p, $x =$ 7,620 м, $v =$ 346,806 м/с	success, x_p, $x =$ 15,240 м, $v =$ 386,344 м/с	—	incomplete
Граница снизу	success, v_p, $x =$ 3,420 м	—	success, x_p, $x =$ 3,420 м	feasible
Граница сверху	success, v_p, $x =$ 3,417 м	—	success, x_p, $x =$ 3,417 м	infeasible
Увеличенное нарушение	success, v_p, $x =$ 3,350 м	—	success, x_p, $x =$ 3,350 м	infeasible
Недопустимый	success, v_p, $x =$ 2,083 м	—	success, x_p, $x =$ 2,083 м	infeasible
Ошибочный	solver_error, error	—	—	numerical_error

Результаты полноценной проверки физических ограничений представлены в таблице 4; в штраф входят только четыре ограничения формулы (5). Для диагностической точки и ошибочной точки они не вычисляются: первая не получила целевую длину, а вторая не получила расчётных рядов.

Таблица 4 — Результаты проверки ограничений

Сценарий	p_m^{max}, МПа	l_m, м	l_m/d	η_e	$p_{mz,+50}$, МПа
Допустимый	247,677	4,792	31,444	0,231	62,824
Цель не достигнута	—	—	—	—	—
Граница снизу	313,303	3,420	22,442	0,266	99,9095
Граница сверху	313,571	3,417	22,418	0,266	100,0521
Увеличенное нарушение	318,458	3,350	21,979	0,268	102,8017
Недопустимый	510,434	2,083	13,665	0,293	167,564
Ошибочный	—	—	—	—	—
Предел	≤ 350	$\leq 7{,}620$	≤ 50	$\leq 0{,}600$	≤ 100

Таблица 5 показывает, что исходный критерий недопустимой точки меньше, чем у допустимой, однако после штрафа порядок меняется. Это подтверждает необходимость хранить исходный критерий, штраф и штрафной критерий раздельно.

Таблица 5 — Критерий, штраф и статус

Сценарий	W_{pm}, **м³**	P, **м³**	W_{pm}^{pen}, **м³**	**Статус**
Допустимый	0,125910	0,000000	0,125910	`feasible`
Цель не достигнута	—	—	—	`incomplete`
Граница снизу	0,099885723	0	0,099885723	`feasible`
Граница сверху	0,099815884	$2{,}716 \cdot 10^{-7}$	0,099816156	`infeasible`
Увеличенное нарушение	0,098544632	0,000784965	0,099329597	`infeasible`
Недопустимый	0,064508	0,666607	0,731115	`infeasible`
Ошибочный	—	1000000	1000000	`numerical_error`

У недопустимой точки нормированные нарушения по максимальному давлению и горячему дульному давлению равны соответственно 0,458384 и 0,675641. Подстановка этих значений в формулу (7) даёт \$0{,}458384^2+0{,}675641^2=0{,}666607\$ м³. Остальные два нарушения равны нулю. Ошибочный сценарий возвращает контролируемый результат и не мешает обработке рассчитанных точек. Для сценария «Цель не достигнута» в таблицах 4 и 5 стоят тире: конечные $x = 7{,}620$ м и $v = 346{,}806$ м/с, а также продолженный диагностический ряд до $x = 15{,}240$ м не заменяют целевую длину и не используются для критерия или горячего дульного давления.

3.2 Анализ скорости

На рисунке 1 показана скорость при +20 °C до первого достижения цели; граница \$50d\$ остаётся защитным пределом базовой ветви. График нужен для проверки достижения 700 м/с и для определения l_m по первому пересечению горизонтальной границы. Сплошная линия с круглыми маркерами соответствует допустимому решению, штриховая линия с квадратными маркерами — недопустимому.

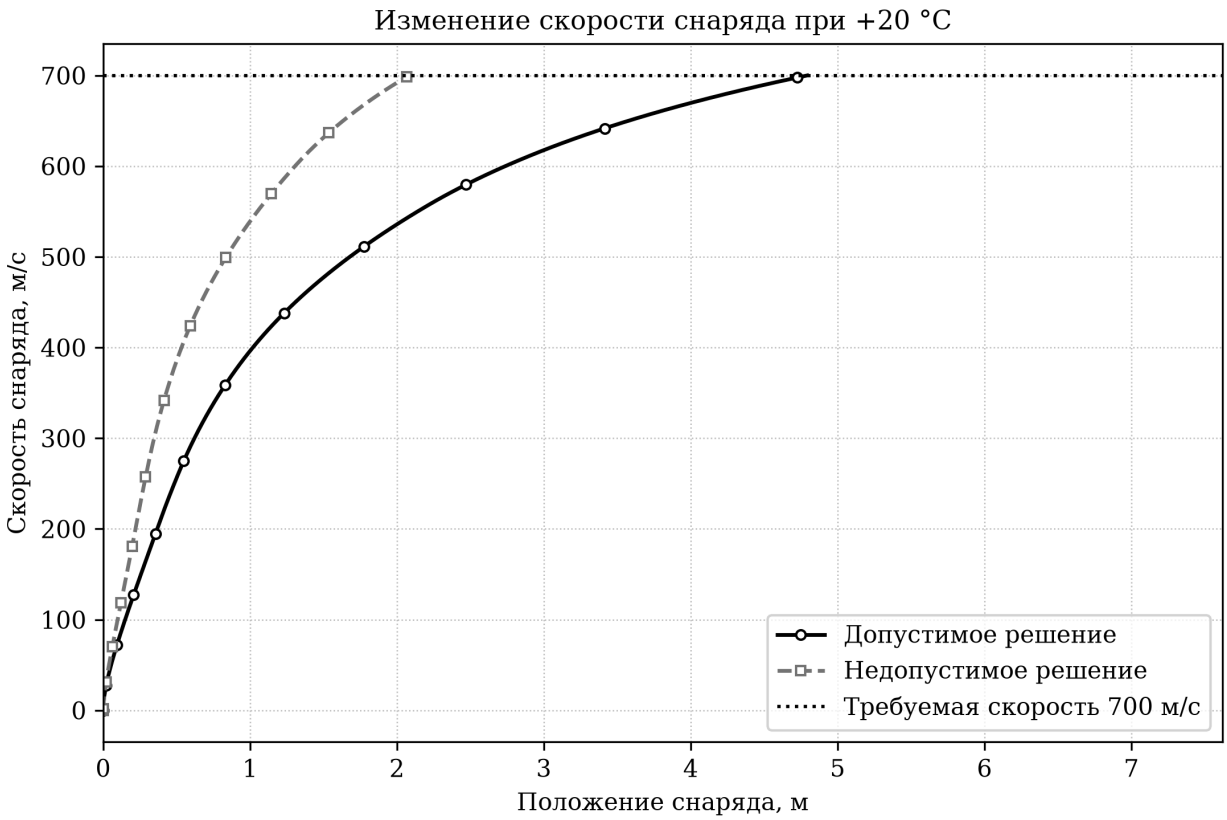


Рисунок 1 — Изменение скорости снаряда при +20 °C

Допустимое решение достигает 700 м/с при 4,792 м, недопустимое — при 2,083 м. Оба решения обеспечивают требуемую скорость до 7,62 м, поэтому ограничение длины выполнено и штрафа не создаёт. Более быстрый разгон недопустимого решения связан с существенно большим давлением и не является признаком общей допустимости.

3.3 Анализ давления

Рисунок 2 сопоставляет среднебаллистическое давление при +20 °C и +50 °C. Верхняя граница 350 МПа применяется к максимуму нормального расчёта, а линия 100 МПа на нижнем поле обозначает допустимое дульное значение горячего расчёта.

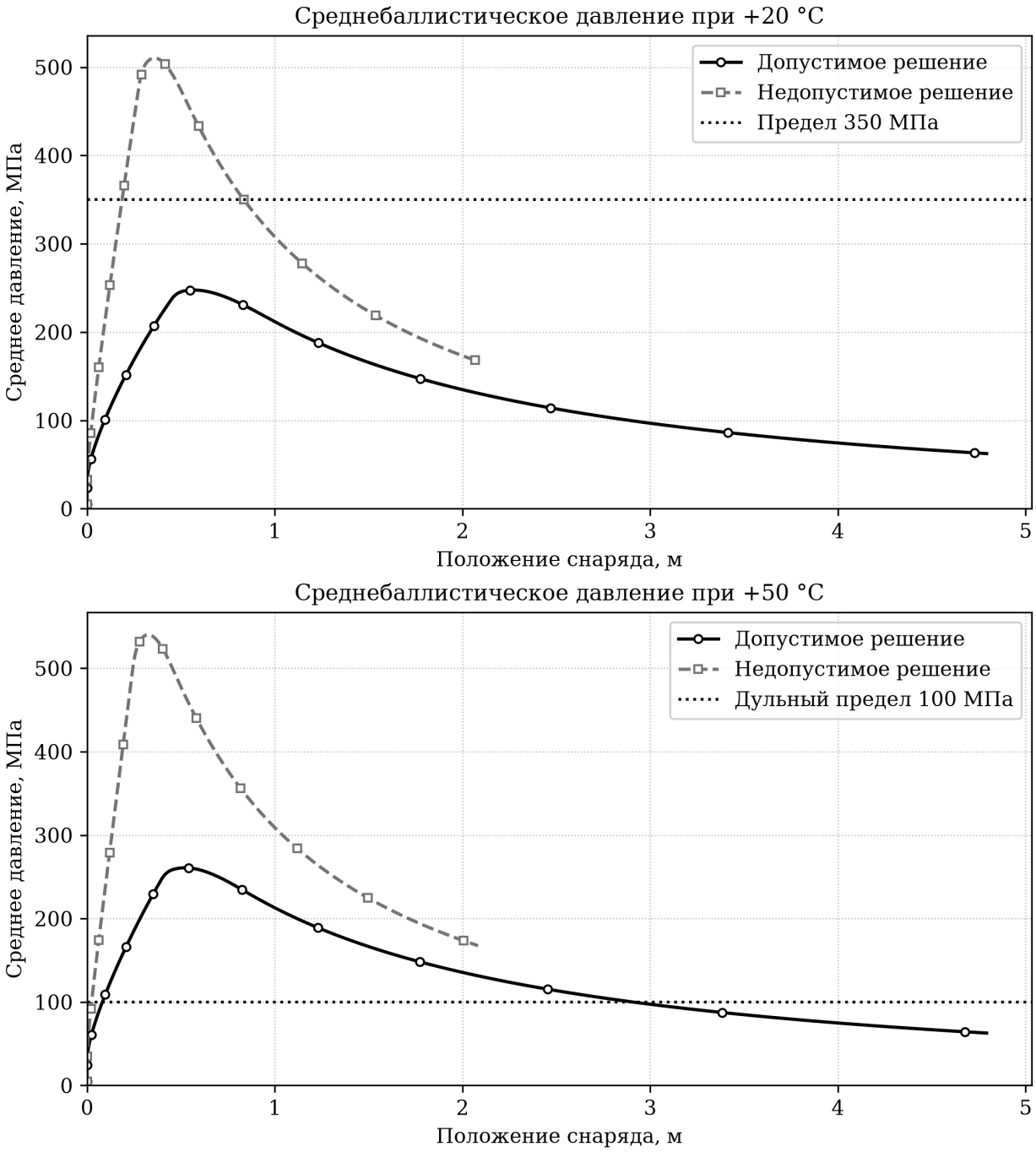


Рисунок 2 — Среднебаллистическое давление при +20 °C и +50 °C

Для допустимого решения максимум при +20 °C составляет 247,677 МПа, а давление при +50 °C на срезе 4,792 м — 62,824 МПа. Запасы равны 102,323 и 37,176 МПа. У недопустимого решения значения 510,434 и 167,564 МПа превышают пределы на 160,434 и 67,564 МПа. Следовательно, его статус `infeasible` обусловлен рассчитанными физическими величинами, а не ошибкой численного решателя.

3.4 Анализ полноты сгорания

На рисунке 3 доля сгоревшего заряда показана в зависимости от относительного положения x/l_m. Такая координата позволяет непосредственно провести вертикальную границу $\eta_e = 0{,}6$ и не смешивать положение конца горения с самой долей сгоревшего пороха.

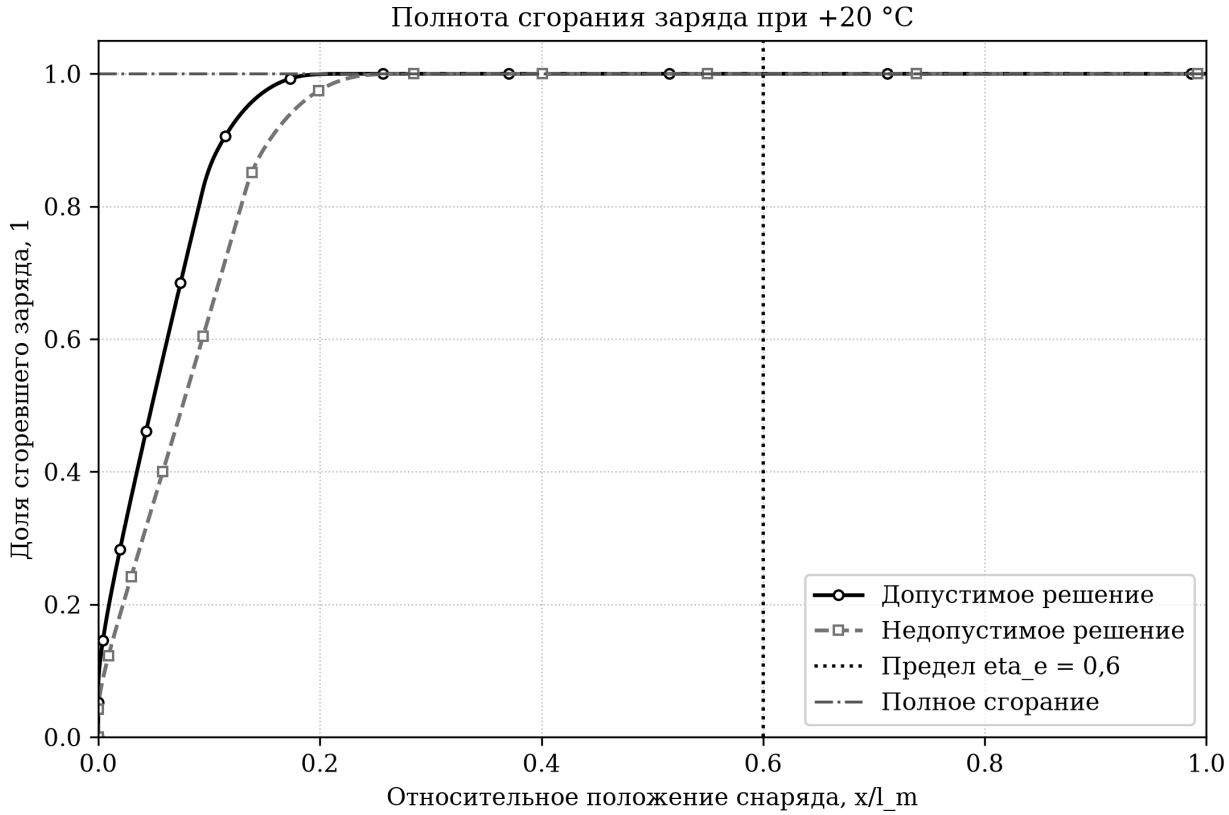


Рисунок 3 — Полнота сгорания заряда при +20 °C

В допустимом решении горение заканчивается при $l_e = 1{,}108$ м и $\eta_e = 0{,}231$. В недопустимом решении получены $l_e = 0{,}610$ м и $\eta_e = 0{,}293$. Оба значения заметно меньше 0,6, поэтому это ограничение выполняется и штрафа не создаёт.

3.5 Совместная проверка ограничений

Рисунок 4 сводит четыре ограничения варианта к форме «допустимо не более единицы»: для каждого условия используется отношение фактического значения к верхнему пределу. Такой график нужен для одновременной проверки всех условий формулы (5) и наглядного объяснения штрафа.



Рисунок 4 — Нормированное использование ограничений

Все столбцы допустимого решения находятся ниже единичной линии. У недопустимого решения столбцы максимального и горячего дульного давления равны 1,46 и 1,68; именно их превышения входят в штраф. Исследование не является поиском оптимума и не заменяет анализ сходимости по сетке. Оно также не охватывает все возможные причины отказа модели: ошибочный сценарий проверяет конкретный случай неизвестной марки пороха.

3.6 Близкая пара и изменение штрафа

На рисунке 5 показана доказательная серия при постоянном объёме каморы 0,035 м³. Верхнее поле фиксирует переход через ограничение горячего дульного давления, нижнее — соответствующее изменение полного штрафа. Остальные три ограничения во всех точках серии выполнены, поэтому наблюдаемый штраф связан только с $p_{mz,+50}$.

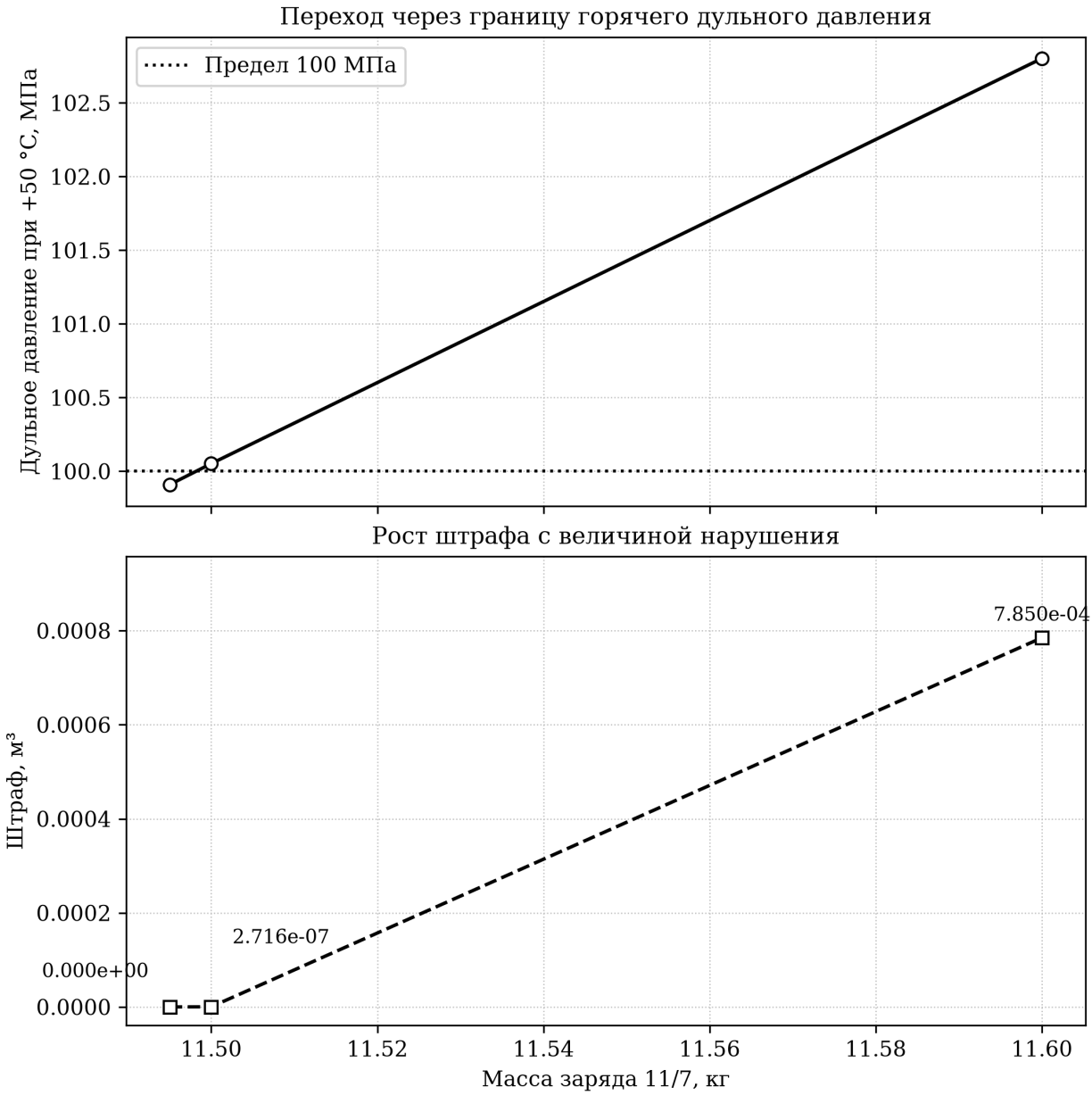


Рисунок 5 — Переход через границу горячего дульного давления и рост штрафа

При увеличении массы заряда с 11,495 до 11,500 кг давление меняется с 99,909508 до 100,052120 МПа. Первая точка имеет запас 0,090492 МПа и нулевой штраф; вторая превышает предел на 0,052120 МПа, её нормированное нарушение равно 0,000521197, а штраф — $2{,}716 \cdot 10^{-7}$ м³. При массе 11,600 кг превышение возрастает до 2,801723 МПа, нормированное нарушение — до 0,0280172, а штраф — до 0,000784965 м³. Таким образом, близкая пара подтверждает срабатывание границы, а третья точка — квадратичный монотонный рост штрафа с величиной нарушения.

3.7 Границы доказательного набора

В выбранной фиксированной области не показано отдельное срабатывание ограничений l_m и η_e: у всех пяти полноценных физических точек из таблицы 4 длина меньше $50d$, а $\eta_e < 0{,}6$. Специальный сценарий с массой 2,0 кг не является примером нарушения длины: цель не достигнута до защитного среза, поэтому он намеренно остаётся статусом `incomplete` без подмены длины конечной координатой. Выбранный набор доказывает вычисление и отображение этих двух условий на представленных точках, но не доказывает срабатывание каждого из них. Дополнительный поиск таких примеров не выполнялся, поскольку для исправления логики целевой и горячей ветвей он не требуется и лабораторная работа не предполагает оптимизацию.

ЗАКЛЮЧЕНИЕ

Реализован прямой расчёт варианта № 17 с применением квазиодномерной модели `ozvb_lagrange` библиотеки `pyballistics` 1.2.1. Программа разделяет целевой расчёт при +20 °C и дополнительную диагностику: l_m, W_{pm} и горячая ветвь допустимы только после фактического останова по $v_p = 700$ м/с. Горячая ветвь дополнительно проверяется по причине останова и конечной координате общего среза.

Проверка семи точек показала ожидаемые результаты. Допустимое решение имеет нулевой штраф и критерий 0,125910 м³. Недопустимое решение нарушает два ограничения по давлению; его штраф 0,666607 м³ повышает критерий с 0,064508 до 0,731115 м³. Близкая пара 99,909508/100,052120 МПа подтверждает переход через выбранную границу, а точка 102,801723 МПа демонстрирует увеличение штрафа до 0,000784965 м³. Неизвестная марка пороха преобразуется в статус `numerical_error` со специальным штрафом и не прерывает обработку остальных точек. Сценарий с массой 2,0 кг получает `incomplete`: сохранены оба диагностических ряда, но критерий, штраф и горячая классификация отсутствуют.

Пять автоматически формируемых рисунков согласуются с табличными данными и показывают границы 700 м/с, 350 МПа, 100 МПа и $\eta_e = 0{,}6$. Повторные запуски сформировали побайтово одинаковые PNG. В выбранном наборе не показано срабатывание l_m и η_e; этот предел доказательности сформулирован в разделе 3.7. Таким образом, критерий, показанные ограничения, штрафная функция, обработка ошибок и воспроизводимость графического материала проверены в пределах выбранных сценариев.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Быков Н. В., Товарнов М. С., Серпинский О. С. Баллистическое проектирование артиллерийских орудий : учебно-методическое пособие. — Москва : Изд-во МГТУ им. Н. Э. Баумана, 2021. — 94 с. — ISBN 978-5-7038-5667-3.
2. PyBallistics 1.2.1 : описание и документация программной библиотеки [Электронный ресурс]. — URL: https://pypi.org/project/pyballistics/1.2.1/ (дата обращения: 25.09.2026).
3. ГОСТ 7.32-2017. Отчёт о научно-исследовательской работе. Структура и правила оформления : текст с поправками [Электронный ресурс]. — URL: https://dd.hse.ru/mirror/pubs/share/902555672 (дата обращения: 25.09.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Текст программы прямого расчёта

```
"""Лабораторная работа № 1: прямой расчёт внутренней баллистики."""

from __future__ import annotations

from dataclasses import dataclass, replace
from enum import StrEnum
from importlib.metadata import version
from math import pi
from numbers import Number
from pathlib import Path
from typing import Any

import matplotlib
import numpy as np
from pyballistics import ozvb_lagrange

matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Исходные данные варианта № 17 в единицах СИ.
CALIBER_M = 0.1524
PROJECTILE_MASS_KG = 43.56
TARGET_VELOCITY_M_S = 700.0
BORE_AREA_M2 = 0.26 * pi * CALIBER_M**2
FORCING_PRESSURE_PA = 30.0e6
FRICTION_FACTOR = 1.02
IGNITION_PRESSURE_PA = 5.0e6
NORMAL_TEMPERATURE_K = 293.15
MAX_BARREL_LENGTH_M = 50.0 * CALIBER_M

DEFAULT_CELL_COUNT = 150
DEFAULT_CFL = 0.9
MAX_MODEL_PRESSURE_PA = 700.0e6
MAX_MODEL_STEPS = 100_000
HOT_TEMPERATURE_K = 323.15
ANALYSIS_STOP_POSITION_M = 2.0 * MAX_BARREL_LENGTH_M

MAX_MEAN_PRESSURE_PA = 350.0e6
MAX_BURN_POSITION_RATIO = 0.6
MAX_HOT_MUZZLE_PRESSURE_PA = 100.0e6
PENALTY_SCALE_M3 = 1.0
NUMERICAL_ERROR_PENALTY_M3 = 1.0e6

FIGURE_DIRECTORY = Path("figures")
FIGURE_DPI = 300
SCENARIO_LINESTYLES = ("-", "--", "-.", ":")
SCENARIO_MARKERS = ("o", "s", "^", "D")
SCENARIO_GRAYS = ("black", "0.45", "0.25", "0.65")
BOUNDARY_CONSTRAINT_NAME = "p_mz(+50 °C)"


@dataclass(frozen=True)
class PowderCharge:
    """Одна навеска табличного пороха."""

    database_name: str
    mass_kg: float


@dataclass(frozen=True)
class BallisticInput:
    """Явное описание входных данных одного прямого расчёта."""

    name: str
    chamber_volume_m3: float
    charges: tuple[PowderCharge, ...]
    temperature_k: float = NORMAL_TEMPERATURE_K
    stop_position_m: float = MAX_BARREL_LENGTH_M
    cell_count: int = DEFAULT_CELL_COUNT
    cfl: float = DEFAULT_CFL
```

```
class RunStatus(StrEnum):
    """Численный статус, не зависящий от выполнения ограничений варианта."""

    SUCCESS = "success"
    SOLVER_ERROR = "solver_error"
    INVALID_RESULT = "invalid_result"


@dataclass(frozen=True)
class ModelRun:
    """Контролируемый результат вызова ``ozvb_lagrange``."""

    input_data: BallisticInput
    status: RunStatus
    stop_reason: str
    execution_time_s: float
    layers: tuple[dict[str, Any], ...]
    model_options: dict[str, Any]
    message: str

    @property
    def is_successful(self) -> bool:
        return self.status is RunStatus.SUCCESS


@dataclass(frozen=True)
class BallisticSeries:
    """Интегральные характеристики последовательности расчётных слоёв."""

    time_s: np.ndarray
    projectile_position_m: np.ndarray
    projectile_velocity_m_s: np.ndarray
    mean_pressure_pa: np.ndarray
    burned_fraction: np.ndarray


class ConstraintKind(StrEnum):
    UPPER = "upper"
    LOWER = "lower"


@dataclass(frozen=True)
class ConstraintCheck:
    """Результат проверки одного размерного ограничения."""

    name: str
    actual: float
    limit: float
    unit: str
    kind: ConstraintKind
    normalized_violation: float
    margin: float
    note: str = ""

    @property
    def is_satisfied(self) -> bool:
        return self.normalized_violation == 0.0


class EvaluationStatus(StrEnum):
    FEASIBLE = "feasible"
    INFEASIBLE = "infeasible"
    INCOMPLETE = "incomplete"
    NUMERICAL_ERROR = "numerical_error"


@dataclass(frozen=True)
class EvaluationResult:
    """Критерий, ограничения и численный статус баллистического решения."""

    input_data: BallisticInput
    status: EvaluationStatus
    normal_run: ModelRun
    diagnostic_normal_run: ModelRun | None
    hot_run: ModelRun | None
    normal_series: BallisticSeries | None
    diagnostic_normal_series: BallisticSeries | None
    hot_series: BallisticSeries | None
    constraints: tuple[ConstraintCheck, ...]
    raw_criterion: float | None
    penalty: float | None
```

```
    penalized_criterion: float | None
    barrel_length_m: float | None
    burn_end_position_m: float | None
    message: str


@dataclass(frozen=True)
class VerificationScenario:
    """Входные данные сценария и ожидаемая классификация результата."""

    input_data: BallisticInput
    expected_status: EvaluationStatus
    include_in_ballistic_figures: bool = False
    include_in_penalty_series: bool = False


def build_model_options(
    input_data: BallisticInput, *, velocity_stop_m_s: float | None = None
) -> dict[str, Any]:
    """Преобразовать явное описание решения во входной словарь pyballistics."""

    stop_conditions: dict[str, float | int] = {
        "x_p": input_data.stop_position_m,
        "p_max": MAX_MODEL_PRESSURE_PA,
        "steps_max": MAX_MODEL_STEPS,
    }
    if velocity_stop_m_s is not None:
        stop_conditions["v_p"] = velocity_stop_m_s

    return {
        "powders": [
            {"omega": charge.mass_kg, "dbname": charge.database_name}
            for charge in input_data.charges
        ],
        "init_conditions": {
            "q": PROJECTILE_MASS_KG,
            "d": CALIBER_M,
            "W_0": input_data.chamber_volume_m3,
            "T_0": input_data.temperature_k,
            "phi_1": FRICTION_FACTOR,
            "p_0": FORCING_PRESSURE_PA,
            "S": BORE_AREA_M2,
        },
        "igniter": {"p_ign_0": IGNITION_PRESSURE_PA},
        "meta_lagrange": {
            "n_cells": input_data.cell_count,
            "CFL": input_data.cfl,
        },
        "stop_conditions": stop_conditions,
    }


def _validate_layers(layers: object) -> tuple[dict[str, Any], ...]:
    """Проверить структуру слоёв и конечность всех численных данных."""

    if not isinstance(layers, list) or not layers:
        raise ValueError("модель не вернула расчётные слои")

    required_keys = {"t", "step_count", "x", "u", "T", "rho", "p", "T_w", "k"}
    checked_layers: list[dict[str, Any]] = []
    for layer_index, layer in enumerate(layers):
        if not isinstance(layer, dict):
            raise TypeError(f"слой {layer_index} не является словарём")
        missing = required_keys.difference(layer)
        if missing:
            raise ValueError(
                f"в слое {layer_index} отсутствуют поля: {', '.join(sorted(missing))}"
            )

        for key, value in layer.items():
            if isinstance(value, np.ndarray):
                if value.size == 0:
                    raise ValueError(f"пустой массив {key!r} в слое {layer_index}")
                if not np.issubdtype(value.dtype, np.number):
                    raise TypeError(f"массив {key!r} в слое {layer_index} нечисловой")
                if not np.all(np.isfinite(value)):
                    raise ValueError(
                        f"NaN или бесконечность в поле {key!r} слоя {layer_index}"
                    )
            elif isinstance(value, Number) and not np.isfinite(float(value)):
                raise ValueError(
```

```
                    f"NaN или бесконечность в поле {key!r} слоя {layer_index}"
                )
            checked_layers.append(layer)

    return tuple(checked_layers)


def run_model_safely(
    input_data: BallisticInput, *, velocity_stop_m_s: float | None = None
) -> ModelRun:
    """Безопасно вызвать модель и преобразовать любую ошибку в ``ModelRun``."""

    options = build_model_options(input_data, velocity_stop_m_s=velocity_stop_m_s)
    try:
        raw_result = ozvb_lagrange(options)
    except Exception as error:  # noqa: BLE001 - защитная граница модели
        return ModelRun(
            input_data=input_data,
            status=RunStatus.SOLVER_ERROR,
            stop_reason="exception",
            execution_time_s=0.0,
            layers=(),
            model_options=options,
            message=f"ozvb_lagrange выбросила {type(error).__name__}: {error}",
        )

    if not isinstance(raw_result, dict):
        return ModelRun(
            input_data=input_data,
            status=RunStatus.INVALID_RESULT,
            stop_reason="invalid_result",
            execution_time_s=0.0,
            layers=(),
            model_options=options,
            message="ozvb_lagrange вернула результат, не являющийся словарём",
        )

    stop_reason = str(raw_result.get("stop_reason", "missing"))
    execution_time = float(raw_result.get("execution_time", 0.0))
    if stop_reason == "error":
        return ModelRun(
            input_data=input_data,
            status=RunStatus.SOLVER_ERROR,
            stop_reason=stop_reason,
            execution_time_s=execution_time,
            layers=(),
            model_options=options,
            message=str(raw_result.get("error_message", "неизвестная ошибка модели")),
        )

    try:
        layers = _validate_layers(raw_result.get("layers"))
    except (TypeError, ValueError, OverflowError) as error:
        return ModelRun(
            input_data=input_data,
            status=RunStatus.INVALID_RESULT,
            stop_reason=stop_reason,
            execution_time_s=execution_time,
            layers=(),
            model_options=options,
            message=f"некорректные расчётные данные: {error}",
        )

    return ModelRun(
        input_data=input_data,
        status=RunStatus.SUCCESS,
        stop_reason=stop_reason,
        execution_time_s=execution_time,
        layers=layers,
        model_options=options,
        message="расчёт завершён; ограничения на этом этапе не классифицируются",
    )


def raw_criterion(chamber_volume_m3: float, barrel_length_m: float) -> float:
    """Вычислить чистый критерий варианта без штрафных слагаемых, м³."""

    return chamber_volume_m3 + BORE_AREA_M2 * barrel_length_m


def extract_ballistic_series(run: ModelRun) -> BallisticSeries:
```

```
    """Извлечь временные ряды из успешного результата ``ozvb_lagrange``.

    В квазиодномерной модели давление задано по ячейкам переменной длины.
    Среднебаллистическое давление вычисляется как пространственное среднее
    ``sum(p_i * dx_i) / sum(dx_i)``, то есть как конечно-объёмный аналог
    определения ``p_m = integral(p dx) / x_p`` из методического пособия.
    """

    if not run.is_successful:
        raise ValueError("характеристики доступны только для успешного расчёта")

    charge_masses = np.asarray(
        [charge.mass_kg for charge in run.input_data.charges], dtype=float
    )
    total_charge_mass = float(np.sum(charge_masses))
    if total_charge_mass <= 0.0:
        raise ValueError("суммарная масса заряда должна быть положительной")

    time: list[float] = []
    position: list[float] = []
    velocity: list[float] = []
    mean_pressure: list[float] = []
    burned_fraction: list[float] = []

    for layer_index, layer in enumerate(run.layers):
        x = np.asarray(layer["x"], dtype=float)
        p = np.asarray(layer["p"], dtype=float)
        dx = np.diff(x)
        if len(p) != len(dx) or np.any(dx <= 0.0):
            raise ValueError(
                f"некорректная пространственная сетка в слое {layer_index}"
            )

        charge_burn = []
        for charge_index in range(1, len(charge_masses) + 1):
            key = f"psi_{charge_index}"
            if key not in layer:
                raise ValueError(f"в слое {layer_index} отсутствует поле {key!r}")
            charge_burn.append(float(np.mean(np.asarray(layer[key], dtype=float))))

        time.append(float(layer["t"]))
        position.append(float(x[-1]))
        velocity.append(float(np.asarray(layer["u"], dtype=float)[-1]))
        mean_pressure.append(float(np.sum(p * dx) / np.sum(dx)))
        burned_fraction.append(
            float(np.dot(charge_masses, charge_burn) / total_charge_mass)
        )

    arrays = tuple(
        np.asarray(values, dtype=float)
        for values in (time, position, velocity, mean_pressure, burned_fraction)
    )
    if not all(np.all(np.isfinite(values)) for values in arrays):
        raise ValueError("извлечённые характеристики содержат NaN или бесконечность")

    return BallisticSeries(*arrays)


def _first_crossing(
    abscissa: np.ndarray, ordinate: np.ndarray, level: float
) -> tuple[float, int] | None:
    """Линейно интерполировать абсциссу первого пересечения уровня снизу."""

    indices = np.flatnonzero(ordinate >= level)
    if indices.size == 0:
        return None
    right = int(indices[0])
    if right == 0:
        return float(abscissa[0]), right
    left = right - 1
    denominator = float(ordinate[right] - ordinate[left])
    fraction = (
        0.0
        if abs(denominator) < 1.0e-14
        else (level - float(ordinate[left])) / denominator
    )
    fraction = min(1.0, max(0.0, fraction))
    value = float(abscissa[left] + fraction * (abscissa[right] - abscissa[left]))
    return value, right
```

```
def _value_at_position(
    series: BallisticSeries, values: np.ndarray, position_m: float
) -> float:
    """Интерполировать временной ряд по положению снаряда."""

    positions = series.projectile_position_m
    if position_m <= positions[0]:
        return float(values[0])
    if position_m >= positions[-1]:
        return float(values[-1])
    return float(np.interp(position_m, positions, values))


def _maximum_before_position(
    series: BallisticSeries, values: np.ndarray, position_m: float
) -> float:
    """Найти максимум ряда до заданного положения включительно."""

    positions = series.projectile_position_m
    preceding_values = values[positions < position_m]
    value_at_position = _value_at_position(series, values, position_m)
    if preceding_values.size == 0:
        return value_at_position
    return max(float(np.max(preceding_values)), value_at_position)


def _upper_constraint(
    name: str, actual: float, limit: float, unit: str, note: str = ""
) -> ConstraintCheck:
    return ConstraintCheck(
        name=name,
        actual=actual,
        limit=limit,
        unit=unit,
        kind=ConstraintKind.UPPER,
        normalized_violation=max(0.0, actual / limit - 1.0),
        margin=limit - actual,
        note=note,
    )


def _lower_constraint(
    name: str, actual: float, limit: float, unit: str, note: str = ""
) -> ConstraintCheck:
    return ConstraintCheck(
        name=name,
        actual=actual,
        limit=limit,
        unit=unit,
        kind=ConstraintKind.LOWER,
        normalized_violation=max(0.0, 1.0 - actual / limit),
        margin=actual - limit,
        note=note,
    )


def calculate_penalty(constraints: tuple[ConstraintCheck, ...]) -> float:
    """Вернуть штраф в м³ по сумме квадратов нормированных нарушений."""

    return PENALTY_SCALE_M3 * sum(
        constraint.normalized_violation**2 for constraint in constraints
    )


def _numerical_error_evaluation(
    input_data: BallisticInput,
    normal_run: ModelRun,
    message: str,
    *,
    diagnostic_normal_run: ModelRun | None = None,
    hot_run: ModelRun | None = None,
    normal_series: BallisticSeries | None = None,
    diagnostic_normal_series: BallisticSeries | None = None,
    hot_series: BallisticSeries | None = None,
    criterion: float | None = None,
    barrel_length_m: float | None = None,
) -> EvaluationResult:
    penalized = (criterion or 0.0) + NUMERICAL_ERROR_PENALTY_M3
    return EvaluationResult(
        input_data=input_data,
        status=EvaluationStatus.NUMERICAL_ERROR,
```

```
        normal_run=normal_run,
        diagnostic_normal_run=diagnostic_normal_run,
        hot_run=hot_run,
        normal_series=normal_series,
        diagnostic_normal_series=diagnostic_normal_series,
        hot_series=hot_series,
        constraints=(),
        raw_criterion=criterion,
        penalty=NUMERICAL_ERROR_PENALTY_M3,
        penalized_criterion=penalized,
        barrel_length_m=barrel_length_m,
        burn_end_position_m=None,
        message=message,
    )


def _incomplete_evaluation(
    input_data: BallisticInput,
    normal_run: ModelRun,
    message: str,
    *,
    normal_series: BallisticSeries | None = None,
    diagnostic_normal_run: ModelRun | None = None,
    diagnostic_normal_series: BallisticSeries | None = None,
    hot_run: ModelRun | None = None,
    hot_series: BallisticSeries | None = None,
    criterion: float | None = None,
    barrel_length_m: float | None = None,
) -> EvaluationResult:
    """Вернуть диагностический результат без полной проверки ограничений.

    Такой результат означает, что базовая цель не достигнута или горячий
    расчёт не дошёл до общего среза. Ряды сохраняются для диагностики, но
    критерий, штраф и дульная классификация не подменяются доступной конечной
    точкой.
    """

    return EvaluationResult(
        input_data=input_data,
        status=EvaluationStatus.INCOMPLETE,
        normal_run=normal_run,
        diagnostic_normal_run=diagnostic_normal_run,
        hot_run=hot_run,
        normal_series=normal_series,
        diagnostic_normal_series=diagnostic_normal_series,
        hot_series=hot_series,
        constraints=(),
        raw_criterion=criterion,
        penalty=None,
        penalized_criterion=None,
        barrel_length_m=barrel_length_m,
        burn_end_position_m=None,
        message=message,
    )


def _ends_at_position(series: BallisticSeries, position_m: float) -> bool:
    """Проверить, что ряд действительно дошёл до требуемого среза."""

    return bool(
        np.isclose(
            series.projectile_position_m[-1],
            position_m,
            rtol=1.0e-9,
            atol=1.0e-8,
        )
    )


def evaluate_solution(input_data: BallisticInput) -> EvaluationResult:
    """Рассчитать целевую и диагностическую ветви одного решения.

    Базовая ветвь останавливается по первой достигнутой целевой скорости или
    по защитному пределу. Только останов ``v_p`` позволяет определять $l_m$,
    критерий и запускать полноценную горячую проверку. Если цель не достигнута,
    дополнительный расчёт сохраняется исключительно как диагностика.
    """

    normal_target_input = replace(
        input_data,
        temperature_k=NORMAL_TEMPERATURE_K,
```

```
    stop_position_m=MAX_BARREL_LENGTH_M,
)
normal_run = run_model_safely(
    normal_target_input, velocity_stop_m_s=TARGET_VELOCITY_M_S
)
if not normal_run.is_successful:
    return _numerical_error_evaluation(
        input_data,
        normal_run,
        f"ошибка расчёта при +20 °C: {normal_run.message}",
    )

try:
    normal_series = extract_ballistic_series(normal_run)
except (TypeError, ValueError, IndexError) as error:
    return _numerical_error_evaluation(
        input_data,
        normal_run,
        f"ошибка обработки расчёта при +20 °C: {error}",
    )

if normal_run.stop_reason != "v_p":
    diagnostic_input = replace(
        input_data,
        temperature_k=NORMAL_TEMPERATURE_K,
        stop_position_m=max(input_data.stop_position_m, ANALYSIS_STOP_POSITION_M),
    )
    diagnostic_run = run_model_safely(diagnostic_input)
    diagnostic_series: BallisticSeries | None = None
    diagnostic_note: str
    if diagnostic_run.is_successful:
        try:
            diagnostic_series = extract_ballistic_series(diagnostic_run)
            diagnostic_note = (
                "дополнительный расчёт сохранён до "
                f"x={diagnostic_series.projectile_position_m[-1]:.9g} м, "
                f"v={diagnostic_series.projectile_velocity_m_s[-1]:.9g} м/с, "
                f"останов={diagnostic_run.stop_reason}"
            )
        except (TypeError, ValueError, IndexError) as error:
            diagnostic_note = f"дополнительный ряд некорректен: {error}"
    else:
        diagnostic_note = (
            "дополнительный расчёт не получен: "
            f"{diagnostic_run.status}, останов={diagnostic_run.stop_reason}"
        )
    return _incomplete_evaluation(
        input_data,
        normal_run,
        "целевая скорость 700 м/с не достигнута до защитного предела "
        f"x={normal_series.projectile_position_m[-1]:.9g} м; "
        f"v={normal_series.projectile_velocity_m_s[-1]:.9g} м/с; "
        f"останов={normal_run.stop_reason}. {diagnostic_note}",
        normal_series=normal_series,
        diagnostic_normal_run=diagnostic_run,
        diagnostic_normal_series=diagnostic_series,
    )

velocity_crossing = _first_crossing(
    normal_series.projectile_position_m,
    normal_series.projectile_velocity_m_s,
    TARGET_VELOCITY_M_S,
)
if velocity_crossing is None:
    return _numerical_error_evaluation(
        input_data,
        normal_run,
        "модель сообщила останов v_p, но в рядах нет целевой скорости",
        normal_series=normal_series,
    )

barrel_length, _ = velocity_crossing
velocity_note = "длина интерполирована при первом достижении скорости"

criterion = raw_criterion(input_data.chamber_volume_m3, barrel_length)
hot_input = replace(
    input_data,
    temperature_k=HOT_TEMPERATURE_K,
    stop_position_m=barrel_length,
)
hot_run = run_model_safely(hot_input)
```

```
if not hot_run.is_successful:
    return _numerical_error_evaluation(
        input_data,
        normal_run,
        f"ошибка расчёта при +50 °C: {hot_run.message}",
        hot_run=hot_run,
        normal_series=normal_series,
        criterion=criterion,
        barrel_length_m=barrel_length,
    )

try:
    hot_series = extract_ballistic_series(hot_run)
except (TypeError, ValueError, IndexError) as error:
    return _numerical_error_evaluation(
        input_data,
        normal_run,
        f"ошибка обработки расчёта при +50 °C: {error}",
        hot_run=hot_run,
        normal_series=normal_series,
        criterion=criterion,
        barrel_length_m=barrel_length,
    )

if hot_run.stop_reason != "x_p" or not _ends_at_position(hot_series, barrel_length):
    return _incomplete_evaluation(
        input_data,
        normal_run,
        "горячий расчёт не дошёл до общего среза; "
        f"ожидалось x={barrel_length:.9g} м, получено "
        f"x={hot_series.projectile_position_m[-1]:.9g} м, "
        f"останов={hot_run.stop_reason}. Дульное ограничение, штраф и "
        "полная классификация не вычислены.",
        normal_series=normal_series,
        hot_run=hot_run,
        hot_series=hot_series,
    )

burn_crossing = _first_crossing(
    normal_series.projectile_position_m,
    normal_series.burned_fraction,
    1.0 - 1.0e-9,
)
if burn_crossing is None:
    burn_end_position = float(normal_series.projectile_position_m[-1])
    burn_note = "заряд не догорел; значение является нижней оценкой"
else:
    burn_end_position = burn_crossing[0]
    burn_note = "положение интерполировано по полноте сгорания"
burn_position_ratio = burn_end_position / barrel_length

max_mean_pressure = _maximum_before_position(
    normal_series,
    normal_series.mean_pressure_pa,
    barrel_length,
)
hot_muzzle_pressure = _value_at_position(
    hot_series,
    hot_series.mean_pressure_pa,
    barrel_length,
)

constraints = (
    _upper_constraint(
        "p_m_max(+20 °C)",
        max_mean_pressure,
        MAX_MEAN_PRESSURE_PA,
        "Па",
        "пространственно осреднённое давление",
    ),
    _upper_constraint(
        "l_m(+20 °C)",
        barrel_length,
        MAX_BARREL_LENGTH_M,
        "м",
        velocity_note,
    ),
    _upper_constraint(
        "eta_e(+20 °C)",
        burn_position_ratio,
        MAX_BURN_POSITION_RATIO,
```

```
            "1",
            burn_note,
        ),
        _upper_constraint(
            "p_mz(+50 °C)",
            hot_muzzle_pressure,
            MAX_HOT_MUZZLE_PRESSURE_PA,
            "Па",
            "пространственно осреднённое давление на дульном срезе",
        ),
    )
    penalty = calculate_penalty(constraints)
    status = (
        EvaluationStatus.FEASIBLE
        if all(constraint.is_satisfied for constraint in constraints)
        else EvaluationStatus.INFEASIBLE
    )
    return EvaluationResult(
        input_data=input_data,
        status=status,
        normal_run=normal_run,
        diagnostic_normal_run=None,
        hot_run=hot_run,
        normal_series=normal_series,
        diagnostic_normal_series=None,
        hot_series=hot_series,
        constraints=constraints,
        raw_criterion=criterion,
        penalty=penalty,
        penalized_criterion=criterion + penalty,
        barrel_length_m=barrel_length,
        burn_end_position_m=burn_end_position,
        message="ограничения выполнены"
        if status is EvaluationStatus.FEASIBLE
        else ("расчёт выполнен, но нарушено одно или несколько ограничений"),
    )


def verify_penalty_properties() -> None:
    """Проверить нулевой штраф и его монотонность по величине нарушения."""

    feasible = (_upper_constraint("test", 9.0, 10.0, "1"),)
    small_violation = (_upper_constraint("test", 11.0, 10.0, "1"),)
    large_violation = (_upper_constraint("test", 12.0, 10.0, "1"),)
    assert calculate_penalty(feasible) == 0.0
    assert 0.0 < calculate_penalty(small_violation) < calculate_penalty(large_violation)


def build_verification_scenarios() -> tuple[VerificationScenario, ...]:
    """Создать основной и доказательный набор проверочных точек."""

    return (
        VerificationScenario(
            input_data=BallisticInput(
                name="Допустимое решение",
                chamber_volume_m3=0.035,
                charges=(PowderCharge(database_name="11/7", mass_kg=10.0),),
            ),
            expected_status=EvaluationStatus.FEASIBLE,
            include_in_ballistic_figures=True,
        ),
        VerificationScenario(
            input_data=BallisticInput(
                name="Цель не достигнута (диагностика)",
                chamber_volume_m3=0.035,
                charges=(PowderCharge(database_name="11/7", mass_kg=2.0),),
            ),
            expected_status=EvaluationStatus.INCOMPLETE,
        ),
        VerificationScenario(
            input_data=BallisticInput(
                name="Граница снизу",
                chamber_volume_m3=0.035,
                charges=(PowderCharge(database_name="11/7", mass_kg=11.495),),
            ),
            expected_status=EvaluationStatus.FEASIBLE,
            include_in_penalty_series=True,
        ),
        VerificationScenario(
            input_data=BallisticInput(
                name="Граница сверху",
```

```
                chamber_volume_m3=0.035,
                charges=(PowderCharge(database_name="11/7", mass_kg=11.500),),
            ),
            expected_status=EvaluationStatus.INFEASIBLE,
            include_in_penalty_series=True,
        ),
        VerificationScenario(
            input_data=BallisticInput(
                name="Увеличенное нарушение",
                chamber_volume_m3=0.035,
                charges=(PowderCharge(database_name="11/7", mass_kg=11.600),),
            ),
            expected_status=EvaluationStatus.INFEASIBLE,
            include_in_penalty_series=True,
        ),
        VerificationScenario(
            input_data=BallisticInput(
                name="Недопустимое решение",
                chamber_volume_m3=0.025,
                charges=(PowderCharge(database_name="11/7", mass_kg=11.5),),
            ),
            expected_status=EvaluationStatus.INFEASIBLE,
            include_in_ballistic_figures=True,
        ),
        VerificationScenario(
            input_data=BallisticInput(
                name="Ошибочное решение",
                chamber_volume_m3=0.035,
                charges=(
                    PowderCharge(database_name="__unknown_powder__", mass_kg=10.0),
                ),
            ),
            expected_status=EvaluationStatus.NUMERICAL_ERROR,
        ),
    )


def _constraint_by_name(result: EvaluationResult, name: str) -> ConstraintCheck:
    """Найти рассчитанное ограничение по его устойчивому имени."""

    for constraint in result.constraints:
        if constraint.name == name:
            return constraint
    raise ValueError(
        f"в результате {result.input_data.name!r} отсутствует ограничение {name!r}"
    )


def verify_calculated_evidence(
    scenarios: tuple[VerificationScenario, ...],
    results: tuple[EvaluationResult, ...],
) -> None:
    """Проверить близкую пару и рост штрафа на рассчитанных точках."""

    penalty_results = tuple(
        result
        for scenario, result in zip(scenarios, results, strict=True)
        if scenario.include_in_penalty_series
    )
    if len(penalty_results) != 3:
        raise RuntimeError("для доказательной серии требуются ровно три точки")
    if any(result.status not in {EvaluationStatus.FEASIBLE, EvaluationStatus.INFEASIBLE} for
result in penalty_results):
        raise RuntimeError("в доказательной серии есть неполный расчёт")

    constraints = tuple(
        _constraint_by_name(result, BOUNDARY_CONSTRAINT_NAME)
        for result in penalty_results
    )
    below, above, farther = constraints
    below_result, above_result, farther_result = penalty_results
    if not below.actual < below.limit < above.actual < farther.actual:
        raise RuntimeError(
            "точки доказательной серии не образуют переход через выбранную границу"
        )
    if not below_result.penalty == 0.0 < above_result.penalty < farther_result.penalty:
        raise RuntimeError("штраф не возрастает с величиной рассчитанного нарушения")

    for result in (above_result, farther_result):
        violated = tuple(
            item.name for item in result.constraints if not item.is_satisfied
```

```
        )
        if violated != (BOUNDARY_CONSTRAINT_NAME,):
            raise RuntimeError(
                f"точка {result.input_data.name!r} должна изолированно нарушать "
                f"ограничение {BOUNDARY_CONSTRAINT_NAME!r}, получено: {violated}"
            )

    print("\nДоказательная серия у границы p_mz(+50 °C) = 100 МПа:")
    for result, constraint in zip(penalty_results, constraints, strict=True):
        print(
            f"  {result.input_data.name}: p_mz={constraint.actual / 1.0e6:.6f} МПа; "
            f"нарушение={constraint.normalized_violation:.9g}; "
            f"штраф={result.penalty:.9g} м^3"
        )


def verify_incomplete_target_diagnostic(
    scenarios: tuple[VerificationScenario, ...],
    results: tuple[EvaluationResult, ...],
) -> None:
    """Проверить, что недостижение цели не подменяется полным решением."""

    diagnostic_results = tuple(
        result
        for scenario, result in zip(scenarios, results, strict=True)
        if scenario.expected_status is EvaluationStatus.INCOMPLETE
    )
    if len(diagnostic_results) != 1:
        raise RuntimeError("требуется ровно один сценарий неполного целевого расчёта")

    result = diagnostic_results[0]
    if result.status is not EvaluationStatus.INCOMPLETE:
        raise RuntimeError("недостижение цели не выделено отдельным статусом")
    if result.raw_criterion is not None or result.barrel_length_m is not None:
        raise RuntimeError("не достигнутая цель не должна определять l_m или W_pm")
    if result.constraints or result.penalty is not None or result.penalized_criterion is not
None:
        raise RuntimeError("неполный целевой расчёт не должен получать штраф")
    if result.hot_run is not None:
        raise RuntimeError("горячий расчёт нельзя запускать без корректной целевой длины")
    if result.diagnostic_normal_run is None or result.diagnostic_normal_series is None:
        raise RuntimeError("для недостижения цели отсутствует дополнительная диагностика")
    if result.normal_run.stop_reason == "v_p":
        raise RuntimeError("неполный сценарий неожиданно достиг целевой скорости")
    if (
        result.diagnostic_normal_series.projectile_position_m[-1]
        < result.normal_series.projectile_position_m[-1]
    ):
        raise RuntimeError("диагностический расчёт не сохранил продолжение траектории")

    print(
        "\nНеполный целевой расчёт сохранён только как диагностика: "
        f"останов={result.normal_run.stop_reason}; "
        f"x_защ={result.normal_series.projectile_position_m[-1]:.6f} м; "
        f"x_диаг={result.diagnostic_normal_series.projectile_position_m[-1]:.6f} м"
    )


def print_evaluation(result: EvaluationResult) -> None:
    """Вывести полный результат одного проверочного сценария."""

    total_charge_mass = sum(charge.mass_kg for charge in result.input_data.charges)
    charge_description = ", ".join(
        f"{charge.database_name}: {charge.mass_kg:.6g} кг"
        for charge in result.input_data.charges
    )
    raw_criterion_text = (
        "не вычислен"
        if result.raw_criterion is None
        else f"{result.raw_criterion:.9g} м^3"
    )
    penalty_text = (
        "не вычислен"
        if result.penalty is None
        else f"{result.penalty:.9g} м^3"
    )
    penalized_text = (
        "не вычислен"
        if result.penalized_criterion is None
        else f"{result.penalized_criterion:.9g} м^3"
    )
```

```
    def endpoint_text(series: BallisticSeries | None) -> str:
        if series is None:
            return "конечные данные отсутствуют"
        return (
            f"x={series.projectile_position_m[-1]:.9g} м; "
            f"v={series.projectile_velocity_m_s[-1]:.9g} м/с"
        )

    print(f"\n=== {result.input_data.name} ===")
    print(
        f"Входные данные: W_0={result.input_data.chamber_volume_m3:.9g} м^3; "
        f"заряд=[{charge_description}]; "
        f"плотность заряжания="
        f"{total_charge_mass / result.input_data.chamber_volume_m3:.9g} кг/м^3; "
        f"omega/q={total_charge_mass / PROJECTILE_MASS_KG:.9g}"
    )
    print(f"Статус: {result.status}")
    print(
        f"Расчёт +20 °C: {result.normal_run.status}, "
        f"причина остановки={result.normal_run.stop_reason}; "
        f"{endpoint_text(result.normal_series)}"
    )
    if result.diagnostic_normal_run is not None:
        print(
            f"Диагностика +20 °C: {result.diagnostic_normal_run.status}, "
            f"причина остановки={result.diagnostic_normal_run.stop_reason}; "
            f"{endpoint_text(result.diagnostic_normal_series)}"
        )
    if result.hot_run is None:
        print("Расчёт +50 °C: не выполнялся")
    else:
        print(
            f"Расчёт +50 °C: {result.hot_run.status}, "
            f"причина остановки={result.hot_run.stop_reason}; "
            f"{endpoint_text(result.hot_series)}"
        )
    print(f"Исходный критерий: {raw_criterion_text}")
    if result.constraints:
        print("Ограничения:")
        for constraint in result.constraints:
            print(
                f"  {constraint.name}: факт={constraint.actual:.9g} "
                f"{constraint.unit}; предел={constraint.limit:.9g} "
                f"{constraint.unit}; нарушение="
                f"{constraint.normalized_violation:.9g}; "
                f"запас={constraint.margin:.9g} {constraint.unit}"
            )
    else:
        print("Ограничения: не вычислены для неполного или численно ошибочного расчёта")
    print(f"Штраф: {penalty_text}")
    print(f"Штрафной критерий: {penalized_text}")
    print(f"Диагностика: {result.message}")


def _series_until_position(
    series: BallisticSeries, values: np.ndarray, position_m: float
) -> tuple[np.ndarray, np.ndarray]:
    """Обрезать ряд по положению и добавить интерполированную конечную точку."""

    positions = series.projectile_position_m
    mask = positions <= position_m
    selected_positions = positions[mask].copy()
    selected_values = values[mask].copy()
    if position_m < positions[-1] and (
        selected_positions.size == 0
        or not np.isclose(selected_positions[-1], position_m)
    ):
        selected_positions = np.append(selected_positions, position_m)
        selected_values = np.append(
            selected_values,
            _value_at_position(series, values, position_m),
        )
    return selected_positions, selected_values


def _line_parameters(index: int, point_count: int) -> dict[str, Any]:
    """Вернуть различимый в чёрно-белой печати стиль линии сценария."""

    return {
        "color": SCENARIO_GRAYS[index % len(SCENARIO_GRAYS)],
```

```
        "linestyle": SCENARIO_LINESTYLES[index % len(SCENARIO_LINESTYLES)],
        "marker": SCENARIO_MARKERS[index % len(SCENARIO_MARKERS)],
        "markevery": max(1, point_count // 12),
        "linewidth": 1.6,
        "markersize": 4.0,
        "markerfacecolor": "white",
    }


def _save_figure(figure: plt.Figure, filename: str) -> Path:
    """Сохранить рисунок с заданным разрешением и закрыть его."""

    path = FIGURE_DIRECTORY / filename
    figure.savefig(path, dpi=FIGURE_DPI, bbox_inches="tight")
    plt.close(figure)
    return path


def _plot_velocity(results: tuple[EvaluationResult, ...]) -> Path:
    figure, axis = plt.subplots(figsize=(7.2, 4.8), constrained_layout=True)
    for index, result in enumerate(results):
        series = result.normal_series
        if series is None:
            continue
        positions, velocities = _series_until_position(
            series,
            series.projectile_velocity_m_s,
            MAX_BARREL_LENGTH_M,
        )
        axis.plot(
            positions,
            velocities,
            label=result.input_data.name,
            **_line_parameters(index, len(positions)),
        )
    axis.axhline(
        TARGET_VELOCITY_M_S,
        color="black",
        linestyle=":",
        linewidth=1.4,
        label="Требуемая скорость 700 м/с",
    )
    axis.set(
        xlabel="Положение снаряда, м",
        ylabel="Скорость снаряда, м/с",
        title="Изменение скорости снаряда при +20 °C",
        xlim=(0.0, MAX_BARREL_LENGTH_M),
    )
    axis.grid(True, linestyle=":", linewidth=0.6, color="0.75")
    axis.legend()
    return _save_figure(figure, "velocity.png")


def _plot_pressure(results: tuple[EvaluationResult, ...]) -> Path:
    figure, axes = plt.subplots(2, 1, figsize=(7.2, 8.0), constrained_layout=True)
    normal_axis, hot_axis = axes
    for index, result in enumerate(results):
        if result.barrel_length_m is None:
            continue
        if result.normal_series is not None:
            positions, pressures = _series_until_position(
                result.normal_series,
                result.normal_series.mean_pressure_pa,
                result.barrel_length_m,
            )
            normal_axis.plot(
                positions,
                pressures / 1.0e6,
                label=result.input_data.name,
                **_line_parameters(index, len(positions)),
            )
        if result.hot_series is not None:
            positions, pressures = _series_until_position(
                result.hot_series,
                result.hot_series.mean_pressure_pa,
                result.barrel_length_m,
            )
            hot_axis.plot(
                positions,
                pressures / 1.0e6,
                label=result.input_data.name,
```

```
                **_line_parameters(index, len(positions)),
            )

    normal_axis.axhline(
        MAX_MEAN_PRESSURE_PA / 1.0e6,
        color="black",
        linestyle=":",
        linewidth=1.4,
        label="Предел 350 МПа",
    )
    hot_axis.axhline(
        MAX_HOT_MUZZLE_PRESSURE_PA / 1.0e6,
        color="black",
        linestyle=":",
        linewidth=1.4,
        label="Дульный предел 100 МПа",
    )
    normal_axis.set(
        xlabel="Положение снаряда, м",
        ylabel="Среднее давление, МПа",
        title="Среднебаллистическое давление при +20 °C",
    )
    hot_axis.set(
        xlabel="Положение снаряда, м",
        ylabel="Среднее давление, МПа",
        title="Среднебаллистическое давление при +50 °C",
    )
    for axis in axes:
        axis.set_xlim(left=0.0)
        axis.set_ylim(bottom=0.0)
        axis.grid(True, linestyle=":", linewidth=0.6, color="0.75")
        axis.legend()
    return _save_figure(figure, "pressure.png")


def _plot_burned_fraction(results: tuple[EvaluationResult, ...]) -> Path:
    figure, axis = plt.subplots(figsize=(7.2, 4.8), constrained_layout=True)
    for index, result in enumerate(results):
        if result.normal_series is None or result.barrel_length_m is None:
            continue
        positions, fractions = _series_until_position(
            result.normal_series,
            result.normal_series.burned_fraction,
            result.barrel_length_m,
        )
        axis.plot(
            positions / result.barrel_length_m,
            fractions,
            label=result.input_data.name,
            **_line_parameters(index, len(positions)),
        )
    axis.axvline(
        MAX_BURN_POSITION_RATIO,
        color="black",
        linestyle=":",
        linewidth=1.4,
        label="Предел eta_e = 0,6",
    )
    axis.axhline(
        1.0,
        color="0.35",
        linestyle="-.",
        linewidth=1.2,
        label="Полное сгорание",
    )
    axis.set(
        xlabel="Относительное положение снаряда, x/l_m",
        ylabel="Доля сгоревшего заряда, 1",
        title="Полнота сгорания заряда при +20 °C",
        xlim=(0.0, 1.0),
        ylim=(0.0, 1.05),
    )
    axis.grid(True, linestyle=":", linewidth=0.6, color="0.75")
    axis.legend()
    return _save_figure(figure, "burned_fraction.png")


def _constraint_usage(constraint: ConstraintCheck) -> float:
    """Привести ограничение к форме, в которой допустимо значение не более 1."""

    if constraint.kind is ConstraintKind.LOWER:
```

```
        return constraint.limit / constraint.actual
    return constraint.actual / constraint.limit


def _plot_normalized_constraints(results: tuple[EvaluationResult, ...]) -> Path:
    constrained_results = tuple(result for result in results if result.constraints)
    labels = (
        "p_m_max/350",
        "l_m/(50d)",
        "eta_e/0,6",
        "p_mz/100",
    )
    positions = np.arange(len(labels), dtype=float)
    width = 0.36
    figure, axis = plt.subplots(figsize=(8.2, 5.0), constrained_layout=True)
    all_usages: list[float] = []
    for index, result in enumerate(constrained_results):
        usages = [_constraint_usage(item) for item in result.constraints]
        all_usages.extend(usages)
        offset = (index - (len(constrained_results) - 1) / 2.0) * width
        bars = axis.bar(
            positions + offset,
            usages,
            width,
            label=result.input_data.name,
            facecolor="white" if index % 2 == 0 else "0.75",
            edgecolor="black",
            linewidth=1.0,
            hatch="//" if index % 2 == 0 else "xx",
        )
        axis.bar_label(bars, fmt="%.2f", padding=2, fontsize=10)
    axis.axhline(
        1.0,
        color="black",
        linestyle=":",
        linewidth=1.5,
        label="Допустимый уровень",
    )
    axis.set(
        ylabel="Нормированное использование ограничения, 1",
        title="Сопоставление ограничений баллистических решений",
        xticks=positions,
        xticklabels=labels,
        ylim=(0.0, max(1.2, max(all_usages, default=1.0) * 1.18)),
    )
    axis.grid(True, axis="y", linestyle=":", linewidth=0.6, color="0.75")
    axis.legend()
    return _save_figure(figure, "normalized_constraints.png")


def _plot_penalty_growth(results: tuple[EvaluationResult, ...]) -> Path:
    """Показать переход через выбранную границу и рост штрафа."""

    charge_masses = np.asarray(
        [
            sum(charge.mass_kg for charge in result.input_data.charges)
            for result in results
        ]
    )
    constraints = tuple(
        _constraint_by_name(result, BOUNDARY_CONSTRAINT_NAME) for result in results
    )
    pressures_mpa = np.asarray([item.actual / 1.0e6 for item in constraints])
    penalties_m3 = np.asarray([result.penalty for result in results])

    figure, axes = plt.subplots(
        2, 1, figsize=(7.2, 7.2), sharex=True, constrained_layout=True
    )
    pressure_axis, penalty_axis = axes
    pressure_axis.plot(
        charge_masses,
        pressures_mpa,
        color="black",
        linestyle="-",
        marker="o",
        markerfacecolor="white",
        linewidth=1.6,
    )
    pressure_axis.axhline(
        MAX_HOT_MUZZLE_PRESSURE_PA / 1.0e6,
        color="black",
```

```
        linestyle=":",
        linewidth=1.4,
        label="Предел 100 МПа",
    )
    pressure_axis.set(
        ylabel="Дульное давление при +50 °C, МПа",
        title="Переход через границу горячего дульного давления",
    )
    pressure_axis.legend()

    penalty_axis.plot(
        charge_masses,
        penalties_m3,
        color="black",
        linestyle="--",
        marker="s",
        markerfacecolor="white",
        linewidth=1.6,
    )
    annotation_styles = (
        {"xytext": (-10, 14), "ha": "right"},
        {"xytext": (10, 30), "ha": "left"},
        {"xytext": (0, 8), "ha": "center"},
    )
    for mass, penalty, style in zip(
        charge_masses, penalties_m3, annotation_styles, strict=True
    ):
        penalty_axis.annotate(
            f"{penalty:.3e}",
            (mass, penalty),
            xytext=style["xytext"],
            textcoords="offset points",
            ha=style["ha"],
            fontsize=9,
        )
    penalty_axis.set(
        xlabel="Масса заряда 11/7, кг",
        ylabel="Штраф, м³",
        title="Рост штрафа с величиной нарушения",
        ylim=(-0.05 * max(penalties_m3), 1.22 * max(penalties_m3)),
    )
    for axis in axes:
        axis.grid(True, linestyle=":", linewidth=0.6, color="0.75")
    return _save_figure(figure, "penalty_growth.png")


def create_figures(
    ballistic_results: tuple[EvaluationResult, ...],
    penalty_results: tuple[EvaluationResult, ...],
) -> tuple[Path, ...]:
    """Автоматически пересоздать полный комплект баллистических рисунков."""

    plt.rcParams.update(
        {
            "font.family": "DejaVu Serif",
            "font.size": 10,
            "axes.titlesize": 11,
            "axes.labelsize": 10,
            "xtick.labelsize": 10,
            "ytick.labelsize": 10,
            "legend.fontsize": 10,
        }
    )
    FIGURE_DIRECTORY.mkdir(parents=True, exist_ok=True)
    return (
        _plot_velocity(ballistic_results),
        _plot_pressure(ballistic_results),
        _plot_burned_fraction(ballistic_results),
        _plot_normalized_constraints(ballistic_results),
        _plot_penalty_growth(penalty_results),
    )


def main() -> None:
    """Рассчитать и вывести основной и доказательный набор точек."""

    print(f"pyballistics {version('pyballistics')}")
    verify_penalty_properties()
    scenarios = build_verification_scenarios()
    results = tuple(evaluate_solution(scenario.input_data) for scenario in scenarios)
    for result in results:
```

```
            print_evaluation(result)

        mismatches = [
            f"{scenario.input_data.name}: ожидался {scenario.expected_status}, "
            f"получен {result.status}"
            for scenario, result in zip(scenarios, results, strict=True)
            if result.status is not scenario.expected_status
        ]
        if mismatches:
            raise RuntimeError("Ошибки классификации сценариев: " + "; ".join(mismatches))
        verify_calculated_evidence(scenarios, results)
        verify_incomplete_target_diagnostic(scenarios, results)
        ballistic_results = tuple(
            result
            for scenario, result in zip(scenarios, results, strict=True)
            if scenario.include_in_ballistic_figures
        )
        penalty_results = tuple(
            result
            for scenario, result in zip(scenarios, results, strict=True)
            if scenario.include_in_penalty_series
        )
        figure_paths = create_figures(ballistic_results, penalty_results)
        print("\nКлассификация всех сценариев соответствует ожидаемой.")
        print("Рисунки пересозданы:")
        for path in figure_paths:
            print(f"  {path.as_posix()}")


    if __name__ == "__main__":
        main()
```